

Kubernetes の HPA 環境下におけるコンテナ集約の適用

田尾瑞紀

複数サービスを 1 つの Pod に同居させる**部分集約**は通信コストを下げるが、Kubernetes では HPA が Deployment 単位で台数を決めるため、集約すると**スケールの粒度が粗くなり**予約資源が増える。本稿では Online Boutique を用いた実機実験により、集約がスループットと通信コストを改善する一方、この損失の大きさが**通信量とワークロード成分**という独立な 2 軸で決まることを示す。

1 はじめに

マイクロサービスを細分化すると、サービス間の呼び出しがネットワークスタックを経由するため、通信処理のオーバーヘッドが生じる。直接通信するサービスを 1 つの Pod に同居させ、通信を localhost 経由にする**部分集約**はこれを削減する。Wickramanayaka ら [2] は Docker Swarm 環境で、通信量の多いコンテナの適正配置・統合を行い通信量が最大 58.5% 減少することを示した。

この効果を、実運用のデファクトスタンダードである Kubernetes に取り入れたい。しかし中核機能である HPA は Deployment 単位で台数を決めるため、集約すると**スケールの粒度が粗くなる**という問題が生じる。本稿ではこの環境での効果を確認したうえで、相手選びに必要な判断軸を示す。

2 部分集約とオートスケーラの干渉

HPA は Deployment 単位で台数を決めるため、同居したコンテナは**同じ台数でしか複製できず**、その台数は各コンテナが必要とする台数の最大値になる。サービス s の枠 (requests) を $r(s)$ 、単独に必要な台

表 1 2 種類の利用者が到達するサービス

利用者	到達するサービス
buy	全 10 サービス
view	frontend, productcatalog, cart, currency, recommendation, ad

数を $n(s)$ 、同居した Pod の台数を $N = \max_s n(s)$ とすると、余分に確保される資源 Δ は

$$\Delta = \sum (N - n(s)) \times r(s) \quad (1)$$

となる。損は「**引きずられた側の枠**」に比例し、必要台数が揃えば $\Delta = 0$ である。以降この Δ を**粒度損**と呼ぶ。

3 実験環境

対象は Online Boutique [1] (11 サービス, gRPC 通信) である。MicroK8s (単一ノード, 16 コア) 上にデプロイし, k6 を用いて負荷を与えた。

負荷は 2 種類の利用者を模した (表 1)。**buy** は閲覧・カート追加・購入を、**view** は閲覧のみを 1 周とする。**view は checkout・payment・shipping・email に一切到達しない**。以降、1 秒あたり x 周のレートで与える負荷を $\text{buy}(x) \cdot \text{view}(x)$ と表す。

評価指標は、粒度損として**予約枠** (稼働 Pod の requests 合計)、通信コストとして **softirq の CPU 時間**を用いた。

3.1 予備実験: スケールアウト後も効果は残るか

先行研究[2] が示した効果は台数 1 の比較である。HPA 環境では負荷に応じて台数が増え、分離構成では複数の宛先に負荷が分散される。そこで**台数を 4 に固定して HPA を切り、スケールアウト後の状態**と比較した。構成は分離 (1 Pod 1 サービス), frontend・recommendation・productcatalog を同居させた **front3**, 全サービスを同居させた **mega** の 3 つである。全コンテナの枠を 150m に揃えたので予約資源はどの構成も 6.3 コアで一致する (3 回復, buy のみ)。

buy(150)–buy(300) ではいずれも目標レートを達成し、取りこぼしと失敗は 0 であった。その条件で 1 周あたりの softirq は分離に対し front3 で 17–23%, mega で 43–51% 少なく、**台数 4 でも同居させるほど通信コストが下がる**。さらにレートを上げると分離は約 360 周/秒で頭打ちになる一方、同居は約 440 周/秒まで伸び、**同じ予約資源で処理できる量が約 2 割多い**。

4 本実験

式 (1) のとおり粒度損は枠 (requests) に比例するため、枠の選び方が結果を左右する。そこで同居させるサービスの枠は実測から決めた。各サービスを 1 台に固定し HPA を切った状態で負荷を 6 段階 (10–150 周/秒) 与え、CPU 使用量を負荷の 1 次式として最小二乗法で近似し、

$$r = (c_0 + c_1 k)/0.7, \quad k = 100 \quad (2)$$

とした。ここで c_0 はアイドル時の CPU 使用量, c_1 は 1 リクエストあたりの CPU 時間, k は基準負荷 [周/秒] である。これにより、適正化したサービスは同じ負荷で目標利用率に達し、必要台数が揃うはずである。**粒度損は同居させたサービスの枠と台数だけで決まるため、適正化は同居対象に限り、他のサービスはマニフェストの値のままとした**。その上で、buy(100) を一定に保ちつつ view を 0・100・200・300 周/秒の 4 段階で変え、**7 つの組み合わせ**を同じ条件で測った (3 サイクル)。

表 2 組み合わせごとの需要の伸びと粒度損 (buy(100))

固定, view 0 → 300 周/秒)		
組み合わせ	需要の伸び	粒度損
frontend+catalog	2.4 / 2.1 倍	なし
frontend+reco	2.4 / 2.2 倍	なし
checkout+email	1.3 / 1.1 倍	なし
frontend+cart	2.4 / 1.5 倍	あり
catalog+checkout	2.1 / 1.3 倍	ほぼなし
frontend+checkout	2.5 / 1.3 倍	あり
frontend+email	2.5 / 1.1 倍	あり

4.1 どの組み合わせで損が出たか

結果を表 2 に示す。ここで**需要**とは利用率と台数の積、すなわち何台分の負荷がかかっているかを表す。view を 0 から 300 周/秒に増やしたとき、この需要が何倍になるかはサービスによって 1.1–2.5 倍と幅がある。

両者を分けるのは、同居した 2 サービスの**伸びが揃っているかどうか**である。伸びが近い 3 つでは粒度損が生じなかった。一方、伸びが食い違う組み合わせでは損が生じ、view 比率によって**引きずる側と引きずられる側が入れ替わった**。ただし伸びが違っても必要台数が同じ整数に落ちれば損は出ない (catalog+checkout)。伸びの差は損の必要条件であって十分条件ではない。

4.2 損が枠では消せない理由

最も比が動いた frontend+email を詳しく見る (表 3)。frontend は emailservice を呼び出さないため、この組み合わせに通信削減の便益はない。

view(0) では email だけが 2 台を要し、frontend が引きずられて 1 台が無駄になる。view(300) では逆に frontend が 3 台を要し、今度は email が引きずられる。**引きずる側と引きずられる側が入れ替わり、損が 9 倍変化した**。損の比 $1077/119 = 9.05$ は枠の比と一致し、式 (1) が両方向で成立している。

view(100) では両者とも 2 台となり損は消える。しかしこれは一点にすぎない。view を 0 から 300 に増やすと **frontend の需要は 2.5 倍**になるのに対し **email は 1.1 倍**にとどまり、両者の**需要比は 2.2 倍変化する**。枠は各サービスに 1 つしか与えられないのに対し需要比は動くため、**全帯で損をゼロにする枠は**

表 3 view による粒度損の変化 (FE+email,
buy(100) 固定, いずれも 3/3 で一致)

view [周/秒]	必要台数 FE/email	需要比 FE/email	粒度損 [mCPU] (引きずられる側)
0	1 / 2	0.85	+1077 (FE)
100	2 / 2	1.24	0
300	3 / 2	1.90	+119 (email)

存在しない。対照として productcatalog と checkout を同居させた構成では、需要比がほとんど動かず粒度損は 12 点中 10 点でゼロであった。

5 考察

便益は通信量で決まり、枠を 3 倍変えても softirq の削減率は 16.4%・16.2% とほぼ変わらない。一方、損は成分で決まる。この独立な 2 軸から、避けるべきは「通信量が多く、かつ成分が異なる」組み合わせとなる。しかし実測では太い通信の両端はいずれも同一成分で、成分をまたぐ通信は最大 2.6% であり、本ア

プリケーションにこの組み合わせは存在しなかった。便益の大きい相手を選ぶ限り、粒度損の危険は踏まないことになる。

6 今後の課題

成分を跨ぐアプリケーションでの実行。

また本実験は単一ノードで行っており、ノードをまたぐ通信 (overlay ネットワーク) を含まない。先行研究[2] が示した効果の大部分は overlay の有無に由来するため、マルチノード環境では便益がさらに大きくなると予想される。

参考文献

- [1] Google Cloud Platform: Online Boutique, GitHub リポジトリ `GoogleCloudPlatform/microservices-demo`, 2025.
- [2] Wickramanayaka, I., Keppitiyagama, C., and Thilakarathna, K.: Communication-Affinity Aware Colocation and Merging of Containers, *International Journal of Advances in ICT for Emerging Regions*, Vol. 15, No. 3(2022).